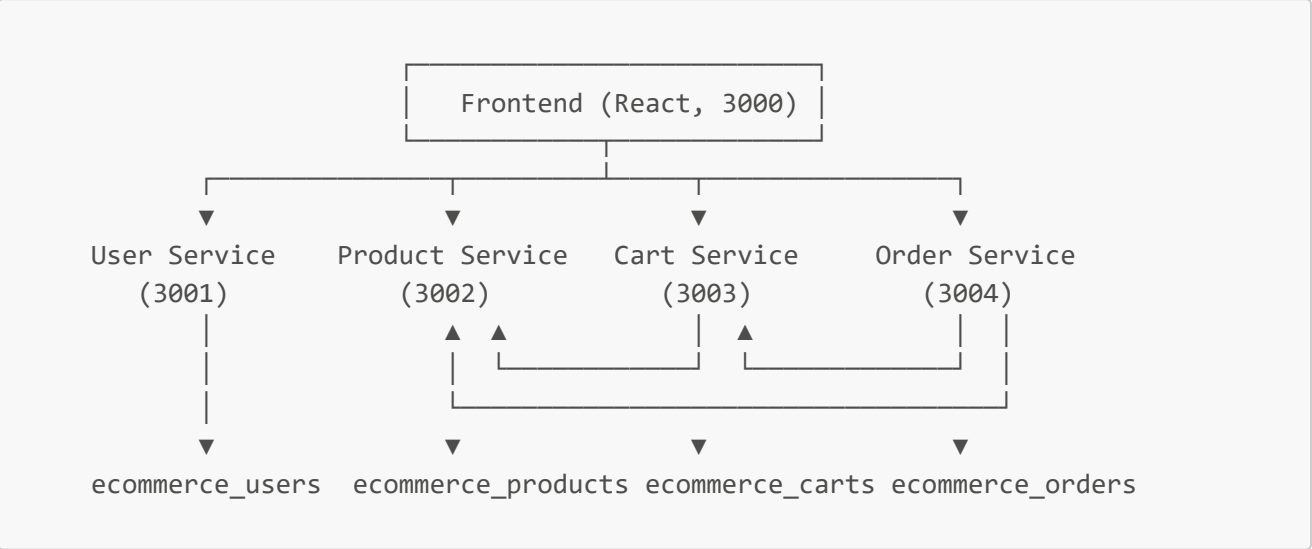
E-Commerce Microservices Application

A full-stack MERN e-commerce application built with microservices architecture, featuring 4 separate Node.js backend services and a React frontend. It ships with a seeded catalog of 30 products, customer and admin accounts, reviews, checkout with simulated payments, and an admin dashboard.

🏗 Architecture Overview

The React frontend calls each service directly over HTTP. Services that need data from another service call it over HTTP as well. Each service owns its own MongoDB database.



- **Cart Service** → **Product Service**: looks up price and stock when items are added or validated
- **Order Service** → **Cart Service**: reads, validates and clears the cart at checkout
- **Order Service** → **Product Service**: reduces stock on order and restores it on cancellation
- **User Service** issues JWTs; the Product and Order services verify them with the shared `JWT_SECRET`

🔧 Technology Stack

Backend

- **Runtime**: Node.js with Express.js
- **Database**: MongoDB with Mongoose ODM
- **Authentication**: JWT tokens with customer / admin roles
- **Architecture**: RESTful APIs with microservices

Frontend

- **Framework**: React 18
- **Routing**: React Router
- **State Management**: React Query + Context API
- **HTTP Client**: Axios
- **Styling**: CSS3 with responsive design

📦 Microservices

1. User Service (Port 3001)

- User registration and authentication
- Profile management
- JWT token generation (tokens carry the user's role)
- User data persistence

Endpoints:

- `POST /api/auth/register` - User registration
- `POST /api/auth/login` - User authentication
- `GET /api/auth/me` - Get current user (logged in)
- `GET /api/users/profile` - Get user profile (logged in)
- `PUT /api/users/profile` - Update name, phone and address (logged in; role, email and password can't be changed here)
- `GET /api/users/:id` - Get user by ID (internal)

2. Product Service (Port 3002)

- Product catalog management
- Category management
- Product search and filtering
- Reviews and ratings
- Inventory tracking

Endpoints:

- `GET /api/products` - Get products. Query params: `page`, `limit`, `category`, `search`, `minPrice`, `maxPrice`, `featured`, `sortBy` (`createdAt`, `price`, `name`, `ratings.average`), `sortOrder`
- `GET /api/products/:id` - Get single product with reviews
- `POST /api/products` - Create product (admin)
- `PUT /api/products/:id` - Update product (admin)
- `DELETE /api/products/:id` - Soft delete product (admin)
- `POST /api/products/:id/reviews` - Add a review (logged in, one per user)
- `POST /api/products/:id/stock` - Adjust stock (internal, used by Order Service)
- `GET /api/categories` - Get all categories
- `GET /api/categories/:id` - Get single category
- `POST /api/categories` - Create category (admin)
- `PUT /api/categories/:id` - Update category (admin)
- `DELETE /api/categories/:id` - Soft delete category (admin)

3. Cart Service (Port 3003)

- Shopping cart management
- Add/remove/update cart items
- Stock checks that include quantities already in the cart

- Cart validation (stock, price changes, removed products)

Endpoints:

- `GET /api/cart/:userId` - Get user's cart
- `POST /api/cart/:userId/items` - Add item to cart
- `PUT /api/cart/:userId/items/:productId` - Update cart item
- `DELETE /api/cart/:userId/items/:productId` - Remove cart item
- `DELETE /api/cart/:userId` - Clear entire cart
- `POST /api/cart/:userId/validate` - Validate cart items

4. Order Service (Port 3004)

- Order creation and management
- Shipping and tax calculation
- Payment processing simulation
- Order status tracking
- Integration with Cart and Product Services

Endpoints:

- `GET /api/orders` - Get all orders + revenue stats (admin)
- `GET /api/orders/user/:userId` - Get user's orders (owner or admin)
- `GET /api/orders/:id` - Get single order (owner or admin)
- `POST /api/orders` - Create new order from the logged-in user's cart
- `POST /api/orders/quote` - Preview shipping, tax and total
- `PUT /api/orders/:id/status` - Update order status (admin)
- `DELETE /api/orders/:id` - Cancel order before it ships (owner or admin)
- `POST /api/payments/process` - Process payment (owner or admin, simulated)
- `POST /api/payments/refund` - Process refund (admin)
- `GET /api/payments/order/:orderId` - Get payment details (owner or admin)

> Routes marked logged in, owner or admin require an `Authorization: Bearer <token>` header from the User Service. The user, product and order services must share the same `JWT_SECRET`.

Pricing Rules

- **Shipping**: Standard is free over $50, otherwise $9.99. Express $19.99, Overnight $39.99
- **Tax**: CA 8.75%, NY 8%, TX 6.25%, FL 6%, all other states 5%

🚀 Getting Started

Prerequisites

- Node.js 16+ and npm
- MongoDB, either running locally on `mongodb://localhost:27017` (the default) or a MongoDB Atlas cluster (see Using MongoDB Atlas)

1. Install dependencies

From the project root:

```
cd backend/user-service && npm install
cd ../product-service && npm install
cd ../cart-service && npm install
cd ../order-service && npm install
cd ../../frontend && npm install
```

2. Load sample data

From the project root:

```
cd backend/user-service && npm run seed      # 2 demo accounts
cd ../product-service && npm run seed        # 6 categories, 30 products
```

> The product seed **replaces** all products and categories in its database. Demo users are only created if they don't exist yet.

3. Run the application

Start each one in its own terminal, from the project root:

```
cd backend/user-service && npm run dev      # 3001
cd backend/product-service && npm run dev   # 3002
cd backend/cart-service && npm run dev      # 3003
cd backend/order-service && npm run dev     # 3004
cd frontend && npm start                    # 3000
```

Then open http://localhost:3000.

`npm run dev` restarts a service automatically when its code changes (nodemon). Use `npm start` to run it once with plain Node.

Demo Accounts

Role	Email	Password
Customer	`demo@shop.com`	`demo123`
Admin	`admin@shop.com`	`admin123`

The login page has buttons that fill these in. The admin account sees an **Admin** link in the header.

Test Payments

Payments are simulated. Any 16-digit card number succeeds, except `4000 0000 0000 0002`, which is always declined (useful for testing the "Retry Payment" flow). Cash on delivery orders are marked paid when an admin marks them delivered.

Environment Variables (optional)

Every value below has a working default for local development. Create a `.env` file in a service's folder only to override them. `.env` files are git-ignored.

backend/user-service/.env:

```
PORT=3001
MONGODB_URI=mongodb://localhost:27017/ecommerce_users
JWT_SECRET=your-jwt-secret-key
```

backend/product-service/.env:

```
PORT=3002
MONGODB_URI=mongodb://localhost:27017/ecommerce_products
JWT_SECRET=your-jwt-secret-key
```

backend/cart-service/.env:

```
PORT=3003
MONGODB_URI=mongodb://localhost:27017/ecommerce_carts
PRODUCT_SERVICE_URL=http://localhost:3002
```

backend/order-service/.env:

```
PORT=3004
MONGODB_URI=mongodb://localhost:27017/ecommerce_orders
JWT_SECRET=your-jwt-secret-key
CART_SERVICE_URL=http://localhost:3003
PRODUCT_SERVICE_URL=http://localhost:3002
USER_SERVICE_URL=http://localhost:3001
```

frontend/.env:

```
REACT_APP_USER_SERVICE_URL=http://localhost:3001
REACT_APP_PRODUCT_SERVICE_URL=http://localhost:3002
REACT_APP_CART_SERVICE_URL=http://localhost:3003
REACT_APP_ORDER_SERVICE_URL=http://localhost:3004
```

> `JWT_SECRET` must be identical in the user, product and order services, because the product and order services verify the tokens the user service issues. If it isn't set, all three fall back to the same built-in development secret, which is fine locally but must be replaced in production.

Using MongoDB Atlas

Set `MONGODB_URI` in each backend service's `.env` to your Atlas connection string, with a different database name per service:

```
# backend/user-service/.env
MONGODB_URI=mongodb+srv://<user>:
<password>@<cluster>.mongodb.net/ecommerce_users?retryWrites=true&w=majority
```

Use `ecommerce_products`, `ecommerce_carts` and `ecommerce_orders` for the other three services. On a shared cluster, add a prefix to the names (for example `myname_ecommerce_users`) so you don't overwrite another app's data. The seed scripts read the same `.env`, so `npm run seed` loads the sample data into Atlas.

> Your machine's IP address must be allowed in the Atlas project's **Network Access** list.

🎯 Features

Customer Features

- **Authentication**: Register and log in with JWT. Protected pages send you back where you were after you log in, and a page refresh keeps you logged in.
- **Product Browsing**: Category chips, partial-match search (header and products page), price filters, featured filter, sorting and pagination, all reflected in the URL
- **Product Pages**: Breadcrumbs, specifications, stock badges, "You may also like" recommendations
- **Reviews & Ratings**: Star ratings and written reviews, one per customer per product
- **Shopping Cart**: Add, update and remove items, with stock checks that include what's already in the cart
- **Checkout**: Saved address prefill, delivery options, live shipping/tax quote from the Order Service, card / PayPal / cash on delivery
- **Order Management**: Order history, cancel before shipping (with stock restored and payment refunded), retry failed payments, "Buy Again"
- **Profile Management**: Update personal information and addresses
- **Notifications**: Toast messages for cart and order actions

Admin Features

- **Dashboard**: Revenue, active orders and orders awaiting fulfilment
- **Order Management**: Filter by status and move orders through confirmed → processing → shipped → delivered, or cancel them
- **Product Management**: Add, edit and remove products; low-stock highlighting

Technical Features

- **Microservices Architecture**: Loosely coupled services, each with its own database
- **RESTful APIs**: Standard HTTP methods and status codes
- **Role-based Access**: Admin-only and owner-only routes enforced in the product and order services
- **Cross-Service Communication**: HTTP-based service interactions
- **Seed Data**: Two commands load a realistic catalog and demo accounts
- **Responsive Design**: Mobile-friendly user interface
- **Error Handling & Loading States**: Comprehensive error management

Project Structure

```
E-CommerceStore/
├── backend/
│   ├── user-service/
│   │   ├── middleware/      # JWT auth + admin check
│   │   ├── models/          # User
│   │   ├── routes/          # auth, users
│   │   ├── seed.js          # demo accounts
│   │   ├── server.js
│   │   └── package.json
│   ├── product-service/
│   │   ├── middleware/      # JWT auth + admin check
│   │   ├── models/          # Product (with reviews), Category
│   │   ├── routes/          # products, categories
│   │   ├── seed.js          # sample catalog
│   │   ├── server.js
│   │   └── package.json
│   ├── cart-service/
│   │   ├── models/          # Cart
│   │   ├── routes/          # cart
│   │   ├── server.js
│   │   └── package.json
│   └── order-service/
│       ├── middleware/      # JWT auth + admin check
│       ├── models/          # Order
│       ├── routes/          # orders, payments
│       ├── server.js
│       └── package.json
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── components/      # Header, Footer, ProductCard, Reviews, StarRating
│   │   ├── contexts/        # Auth, Cart, Toast
│   │   ├── pages/           # Home, Products, ProductDetail, Cart, Checkout,
│   │   │                    # Orders, Profile, Login, Register, Admin,
NotFound
│   │   ├── services/        # API clients for each microservice
│   │   ├── utils/           # formatting helpers
│   │   ├── App.js
```

```
│   │   └── index.js
│   └── package.json
├── LICENSE
└── README.md
```

🔧 API Testing

You can test the APIs using tools like Postman or curl:

```
# Health check for all services
curl http://localhost:3001/health
curl http://localhost:3002/health
curl http://localhost:3003/health
curl http://localhost:3004/health

# Register a new user
curl -X POST http://localhost:3001/api/auth/register \
  -H "Content-Type: application/json" \
  -d
'{"firstName":"John","lastName":"Doe","email":"john@example.com","password":"pa
ssword123"}'

# Log in (copy the "token" from the response)
curl -X POST http://localhost:3001/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"demo@shop.com","password":"demo123"}'

# Call a protected route with the token
curl http://localhost:3001/api/auth/me -H "Authorization: Bearer <token>"

# Search products
curl "http://localhost:3002/api/products?
search=headphones&sortBy=price&sortOrder=asc"

# Get categories
curl http://localhost:3002/api/categories

# Preview shipping and tax
curl -X POST http://localhost:3004/api/orders/quote \
  -H "Content-Type: application/json" \
  -d '{"subtotal":40,"shippingMethod":"standard","state":"CA"}'
```

🚀 Deployment

Production Considerations

1. **Environment Variables**: Use proper environment variable management, and set a strong, shared `JWT_SECRET`

2. **Database**: Use MongoDB Atlas or other managed database services
3. **Process Management**: Use PM2 or similar for process management
4. **Internal Routes**: The cart routes and `POST /api/products/:id/stock` have no authentication because other services call them. Keep them on a private network, or add service-to-service auth, before exposing the services publicly.
5. **CORS**: Services currently accept requests from any origin; restrict this to your frontend's domain
6. **Load Balancing**: Implement load balancing for high availability
7. **Monitoring**: Add logging and monitoring solutions
8. **Security**: Add rate limiting and other security measures

🤝 Contributing

1. Fork the repository
2. Create a feature branch
3. Commit your changes
4. Push to the branch
5. Create a Pull Request

📝 License

This project is licensed under the MIT License.

🆘 Support

For support and questions:

- Check the documentation
- Review API endpoints and expected payloads
- Ensure all services are running
- Verify database connections (and Atlas network access, if you use Atlas)
- Check environment variables, especially that `JWT_SECRET` matches across services

🔮 Future Enhancements

- **API Gateway**: Centralized request routing and authentication
- **Docker Containerization**: Containerize services with docker-compose
- **Message Queues**: Async communication between services
- **Caching**: Redis caching for improved performance
- **Search Engine**: Elasticsearch for advanced product search
- **File Upload**: Image upload and management
- **Email Service**: Order confirmations and notifications
- **Analytics**: Order and user analytics
- **Payment Integration**: Real payment gateway integration